

CENTRO UNIVERSITÁRIO INTERNACIONAL – UNINTER

Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas

Projeto Multidisciplinar – Trilha Back-End

Prof. Me. Luciane Yanase Kanashiro

Pedro Marques Mesquita

RU: 4710499

Polo: EAD – Ensino a Distância

SISTEMA DE PEDIDOS PARA A REDE RAÍZES DO NORDESTE

Atividade Prática de Projeto Multidisciplinar – Trilha Back-End

Trabalho apresentado como Atividade Prática de Projeto Multidisciplinar do curso de Análise e Desenvolvimento de Sistemas do Centro Universitário Internacional – Uninter, na trilha Back-End, como requisito parcial para obtenção de nota.

Semestre: 1.º Semestre de 2026

Brasil – 2026

Repositorio GitHub: <https://github.com/PedroMesquita7/raizes-nordeste-backend.git>

SUMÁRIO

1. ANÁLISE DO PROBLEMA E REQUISITOS

- 1.1 Contexto do Negócio
- 1.2 Requisitos Funcionais (RF)
- 1.3 Requisitos Não Funcionais (RNF)
- 1.4 Multicanalidade
- 1.5 Fidelização, Pagamento Mock e LGPD
- 1.6 Priorização do MVP

2. MODELAGEM E ARQUITETURA DE BACK-END

- 2.1 Diagrama de Casos de Uso
- 2.2 Diagrama Entidade-Relacionamento (DER)
- 2.3 Diagrama de Classes do Domínio
- 2.4 Arquitetura em Camadas
- 2.5 Coerência entre Modelagem e Endpoints
- 2.6 Descrição de Feature — UC01

3. DOCUMENTAÇÃO DOS ENDPOINTS

4. IMPLEMENTAÇÃO DA API E REGRAS DE NEGÓCIO

5. SEGURANÇA, LGPD, LOGS E AUDITORIA

6. PLANO DE TESTES (API)

7. ENTREGA TÉCNICA E DOCUMENTAÇÃO

8. CONCLUSÃO

REFERÊNCIAS

DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

INTRODUÇÃO

A Rede Raízes do Nordeste é uma franquia de alimentação regional em expansão, com unidades físicas distribuídas em diferentes cidades do Nordeste brasileiro. O crescimento da rede traz desafios operacionais relevantes: padronizar o cardápio e os preços-base a partir da matriz, mas permitir que cada unidade gerencie seu próprio estoque; atender clientes por múltiplos canais de atendimento (aplicativo, totem de autoatendimento e balcão presencial); e processar pagamentos de forma integrada e rastreável, com tratamento adequado de dados pessoais dos clientes em conformidade com a Lei Geral de Proteção de Dados (Lei n.º 13.709/2018 – LGPD).

Este trabalho apresenta o projeto e a implementação de uma API REST como solução de Back-End para o sistema de pedidos da rede. O objetivo principal é entregar um MVP funcional que cubra o fluxo crítico de negócio — desde a autenticação do usuário até a criação de pedido, a validação de estoque, o processamento de pagamento simulado e a atualização de situação — de forma documentada, testável e reproduzível.

O projeto foi desenvolvido utilizando Java 21 com Spring Boot 3.2.5, Spring Security com JWT para autenticação, Spring Data JPA com H2 (modo demo) ou PostgreSQL (modo produção) para persistência, Flyway para versionamento de schema e Swagger/OpenAPI para documentação interativa da API. A validação funcional foi realizada por meio de uma coleção Postman com 13 cenários de teste organizados, cobrindo os fluxos positivos e os principais cenários de erro e regra de negócio.

1. ANÁLISE DO PROBLEMA E REQUISITOS

1.1 Contexto do Negócio

A Rede Raízes do Nordeste opera como uma franquia com unidades distribuídas em diversas cidades, controladas por uma matriz que padroniza o cardápio e os preços-base. Cada unidade mantém seu próprio estoque independente e atende clientes por três canais principais: aplicativo mobile, totem de autoatendimento e balcão presencial. O Back-End precisa sustentar o ciclo de vida completo do pedido — criação, consulta, atualização de

situação e pagamento — de forma consistente entre todas as unidades, controlando estoque por unidade e garantindo conformidade com a LGPD.

1.2 Requisitos Funcionais (RF)

Os requisitos funcionais identificados para o MVP são:

RF01 – Cadastrar usuários do sistema com perfil (role) definido, para autenticação e controle de acesso.

RF02 – Autenticar usuários via login (e-mail e senha) e emitir token JWT para uso nas requisições subsequentes.

RF03 – Cadastrar clientes, registrando dados pessoais mínimos (nome, e-mail, telefone, idade) e o aceite LGPD.

RF04 – Disponibilizar catálogo de produtos com preço-base, podendo indicar disponibilidade sazonal (período junino).

RF05 – Controlar estoque de produtos por unidade, de forma independente entre unidades da rede.

RF06 – Criar pedidos vinculados a um cliente, uma unidade, um canal de atendimento e uma lista de itens, calculando o valor total automaticamente.

RF07 – Validar, no momento da criação do pedido, a existência do cliente, da unidade, dos produtos e a disponibilidade de estoque suficiente.

RF08 – Consultar pedidos (listagem geral e busca por ID), incluindo seus itens e situação atual, com filtro por canal.

RF09 – Atualizar a situação do pedido ao longo do fluxo operacional (AGUARDANDO_PAGAMENTO, PAGAMENTO_APROVADO, EM_PREPARO, PRONTO, ENTREGUE, CANCELADO).

RF10 – Processar o pagamento do pedido por meio de um gateway mock, simulando aprovação ou recusa conforme regra de negócio.

RF11 – Impedir o reprocessamento de pagamento de um pedido que já não esteja aguardando pagamento.

1.3 Requisitos Não Funcionais (RNF)

RNF01 – Segurança: autenticação via JWT (stateless) e senhas armazenadas com hash BCrypt.

RNF02 – Padronização de erros: respostas de erro com campo "erro" (código) e "mensagem" (descrição legível).

RNF03 – Documentação: a API expõe contrato via Swagger/OpenAPI em /swagger-ui.html.

RNF04 – Persistência: dados armazenados em H2 (demo) ou PostgreSQL (produção) com versionamento de schema via Flyway.

RNF05 – Testabilidade: o fluxo crítico validável de forma reproduzível via coleção Postman.

RNF06 – Privacidade (LGPD mínimo): coleta de dados pessoais limitada à finalidade do serviço, com registro de consentimento.

1.4 Multicanalidade

O domínio modela a multicanalidade por meio do campo canalPedido (enum CanalAtendimento) na entidade Pedido, com os valores: APP, TOTEM, BALCAO, PICKUP e WEB. Esse campo é obrigatório na criação do pedido e é retornado nas listagens e consultas individuais. A API permite filtrar pedidos por canal via query param (?canalPedido=TOTEM), possibilitando rastreabilidade e análise de desempenho por canal de origem da venda.

1.5 Fidelização, Pagamento Mock e LGPD

Fidelização: não foi implementada nesta entrega, registrada como proposta para iteração futura na forma de uma entidade de pontuação vinculada ao Cliente, acumulada a cada pedido com pagamento aprovado.

Pagamento externo (mock): implementado pela classe GatewayPagamentoMock, que simula a resposta de um gateway real sem integração externa. A regra de simulação aprova pagamentos com valor total até R\$ 500,00 e recusa valores acima desse limite, permitindo evidenciar cenários de sucesso e falha de forma determinística.

LGPD (mínimo): a entidade Cliente possui os campos `aceiteLgpd` (booleano) e `dataAceiteLgpd` (timestamp), registrando o consentimento do titular. Os dados coletados (nome, e-mail, telefone, idade) são minimizados ao necessário. Senhas de usuários são armazenadas com hash BCrypt e nunca retornadas nas respostas da API.

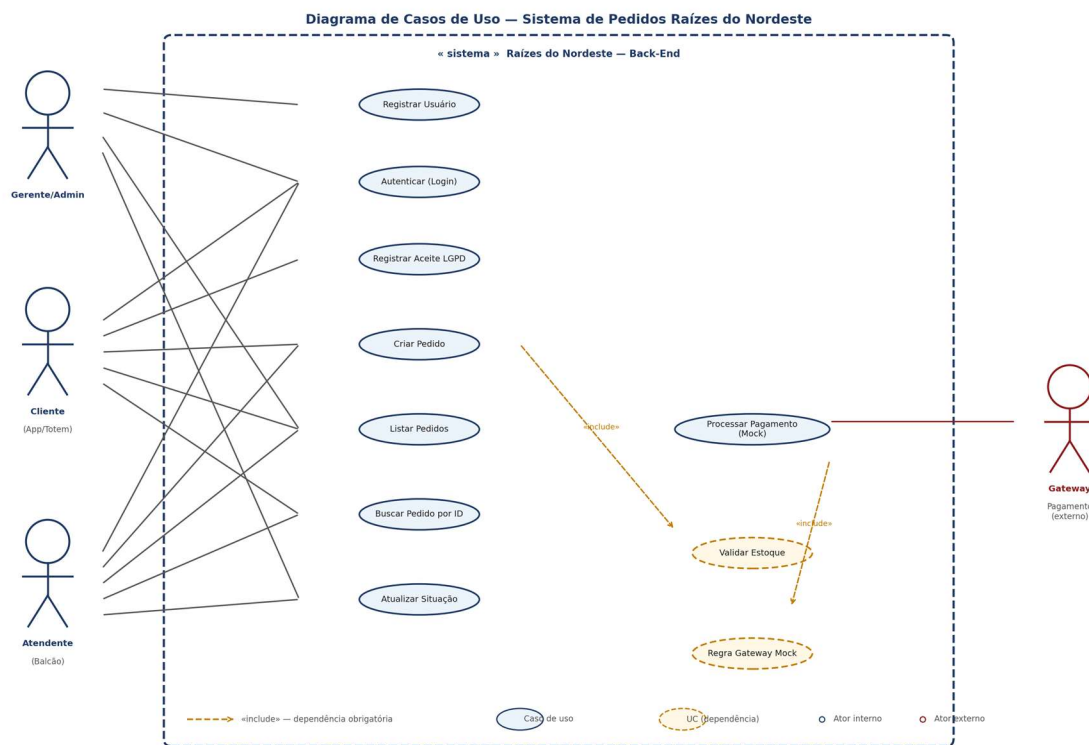
1.6 Priorização do MVP

Implementado: autenticação JWT, cadastro de cliente/produto/unidade/estoque, criação e consulta de pedidos com validação de estoque, atualização de situação, pagamento mock com aprovação e recusa, padrão de erro consistente, documentação Swagger e 13 cenários de teste.

Não implementado (proposta): programa de fidelização, promoções/campanhas, logs/auditoria estruturados e restrição de endpoints por perfil via `@PreAuthorize`.

2. MODELAGEM E ARQUITETURA DE BACK-END

2.1 Diagrama de Casos de Uso



O sistema possui três atores principais: Cliente (App/Totem), Atendente (Balcão) e Gerente/Admin. O Gateway de Pagamento atua como ator externo. Os casos de uso implementados incluem: Registrar usuário, Autenticar (login), Criar pedido, Listar pedidos, Buscar pedido por ID, Atualizar situação do pedido, Processar pagamento (mock) e Registrar aceite LGPD do cliente. As relações de inclusão (include) indicam que a criação de pedido depende da validação interna de estoque, e que o processamento de pagamento depende da regra do gateway mock.

2.2 Diagrama Entidade-Relacionamento (DER)

O modelo de dados é composto por sete entidades principais: usuarios, unidades, clientes, produtos, estoques, pedidos e itens_pedido. Cardinalidades: cada pedido pertence a um cliente e a uma unidade (N:1); cada item de pedido pertence a um pedido e referencia um produto (N:1); cada registro de estoque vincula um produto a uma unidade específica (chave única produto_id + unidade_id), permitindo saldos independentes por loja.

Tabelas e atributos principais:

- **usuarios:** id (PK), nome, email (unique), senha, perfil, ativo, criado_em
- **unidades:** id (PK), nome, cidade, estado, endereco, tipo, ativa
- **clientes:** id (PK), nome, email (unique), telefone, idade, aceite_lgpd, data_aceite_lgpd, criado_em
- **produtos:** id (PK), nome, descricao, preco_base, ativo, disponivel_junino
- **estoques:** id (PK), produto_id (FK), unidade_id (FK), quantidade, atualizado_em
- **pedidos:** id (PK), cliente_id (FK), unidade_id (FK), canal_pedido, situacao, valor_total, forma_pagamento, situacao_pagamento, criado_em
- **itens_pedido:** id (PK), pedido_id (FK), produto_id (FK), quantidade, preco_unitario, subtotal

2.3 Diagrama de Classes do Domínio

As entidades JPA do pacote dominio.modelo refletem fielmente o DER. A classe Pedido centraliza os atributos transacionais (situacao, canalPedido, valorTotal, formaPagamento, situacaoPagamento) e mantém composição com ItemPedido (cascade = ALL). Os enums

SituacaoPedido e CanalAtendimento tipam respectivamente o estado do pedido e o canal de origem da venda. A anotação @JsonIgnore no atributo pedido de ItemPedido previne referência circular na serialização JSON.

2.4 Arquitetura em Camadas

O projeto adota separação em quatro camadas com responsabilidades bem definidas:

api/controller/	Camada API: Controllers REST + ConfiguracaoSwagger
dominio/modelo/	Camada Domain: Entidades JPA do negocio
dominio/enumeracao/	Camada Domain: Enums (SituacaoPedido, CanalAtendimento)
infraestrutura/seguranca/	Infraestrutura: JWT, FiltroJwt, ConfiguracaoSeguranca
infraestrutura/repositorio/	Infraestrutura: Repositorios Spring Data JPA
infraestrutura/pagamento/	Infraestrutura: GatewayPagamentoMock

Essa organização separa claramente a regra de domínio da infraestrutura técnica e da camada de exposição HTTP. A regra de negócio principal reside nos controllers nesta versão MVP — a extração de uma camada de serviços dedicados é indicada como melhoria na Conclusão.

2.5 Coerência entre Modelagem e Endpoints

Caso de uso / Entidade	Endpoint	Regras aplicadas
Registrar usuário	POST /autenticacao/registro	Hash BCrypt; perfil obrigatório
Autenticar	POST /autenticacao/login	Validação de credenciais; emissão JWT
Criar pedido	POST /pedidos	Validação de campos; estoque; existência de entidades
Listar / Buscar pedido	GET /pedidos, GET /pedidos/{id}	Filtro por canal; itens aninhados
Atualizar situação	PATCH /pedidos/{id}/situacao	Transição de SituacaoPedido
Processar pagamento	POST /pagamentos/processar/{id}	Aprovação/recusa por valor; bloqueio reprocessamento

2.6 Descrição de Feature — UC01: Realizar Pedido e Processar Pagamento

Ator principal: Cliente (App/Totem) ou Atendente (Balcão) | Demais atores: Gateway de Pagamento (mock)

Pré-condições: (1) Usuário autenticado com token JWT válido. (2) Cliente, Unidade e Produto(s) com ids existentes e estoque suficiente.

Fluxo principal: (1) POST /pedidos com dados e itens. (2) Validação de campos obrigatórios. (3) Verificação de existência das entidades e estoque. (4) Cálculo do valorTotal. (5) Persistência com AGUARDANDO_PAGAMENTO. (6) POST /pagamentos/processar/{id}. (7) Gateway mock: aprova se valorTotal <= R\$500; recusa caso contrário. (8) Atualização de situação para PAGAMENTO_APROVADO ou CANCELADO.

Regras: RN01 – valorTotal = soma(precoBase x quantidade). RN02 – Gateway mock aprova valores <= R\$500 e recusa superiores. RN03 – Pedido só pode ser pago se AGUARDANDO_PAGAMENTO. RN04 – clienteId, unidadeId, canalPedido, formaPagamento e itens são obrigatórios.

3. DOCUMENTAÇÃO DOS ENDPOINTS

Todos os endpoints, exceto /autenticacao/registro e /autenticacao/login, exigem token JWT no header Authorization: Bearer {token}. Contrato completo disponível em <http://localhost:8080/swagger-ui.html>.

Padrão de erro em toda a API:

```
{ "erro": "NOME_DO_ERRO", "mensagem": "Descricao legivel" }
```

3.1 Auth — Registrar Usuário

POST /autenticacao/registro | público

```
{ "nome": "Pedro Marques", "email": "pedro@raizes.com", "senha": "senha123", "perfil": "ADMIN" }
```

Response (201): { "mensagem": "Usuario cadastrado com sucesso." }

3.2 Auth — Login

POST /autenticacao/login | público

```
{ "email": "pedro@raizes.com", "senha": "senha123" }  
Response (200): { "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "perfil": "ADMIN" }
```

3.3 Pedidos — Criar Pedido (fluxo crítico)

POST /pedidos | JWT

```
{
  "clienteId":1,"unidadeId":1,"canalPedido":"APP","formaPagamento":"PIX","itens":[{"produtoId":1,"quantidade":2}] }
Response (201): {
  "pedidoId":1,"situacao":"AGUARDANDO_PAGAMENTO","canalPedido":"APP","valorTotal":37.80,"situacaoPagamento":"PENDENTE" }
```

3.4 Pedidos — Listar / Buscar

GET /pedidos?canalPedido=APP | JWT → 200 + lista. GET /pedidos/{id} | JWT → 200 + objeto completo com itens aninhados.

3.5 Pedidos — Atualizar Situação

PATCH /pedidos/{id}/situacao | JWT

```
Request: { "situacao":"EM_PREPARO" } | Response: {
  "pedidoId":1,"novaSituacao":"EM_PREPARO" }
```

3.6 Pagamentos — Processar Pagamento (mock)

POST /pagamentos/processar/{id} | JWT

```
Aprovado: {
  "situacaoPagamento":"APROVADO","pedidoId":1,"novaSituacaoPedido":"PAGAMENTO_APROVADO","mensagem":"Pagamento aprovado com sucesso.","idTransacao":"uuid" }
Recusado: {
  "situacaoPagamento":"RECUSADO","novaSituacaoPedido":"CANCELADO","mensagem":"...valor acima do limite..." }
```

Erro 409 SITUACAO_INVALIDA: pedido já processado (bloqueio de reprocessamento).

4. IMPLEMENTAÇÃO DA API E REGRAS DE NEGÓCIO

4.1 Tecnologias Utilizadas

- Linguagem: Java 21 (OpenJDK JBR 21.0.7 — JetBrains Runtime)
- Framework: Spring Boot 3.2.5 (Web, Data JPA, Security, Validation)
- Banco de dados: H2 em memória (demo) ou PostgreSQL (produção) | Flyway para migrations
- Autenticação: JWT (jjwt 0.12.5) via ServicoJwt e FiltroJwt
- Documentação: Swagger/OpenAPI via springdoc-openapi 2.5.0

- Build: Apache Maven 3.9.6

4.2 Fluxo Crítico Ponta a Ponta

O fluxo crítico implementado é: criação de pedido → validação de estoque → processamento de pagamento mock → atualização de situação. Validado em execução real com H2:

1. POST /autenticacao/login → token JWT gerado
2. POST /pedidos (clienteId=1, produtoId=1, qtd=2) → pedidoId=1, valorTotal=37.80, AGUARDANDO_PAGAMENTO
3. POST /pagamentos/processar/1 → APROVADO (R\$37,80 <= R\$500,00), UUID gerado
4. POST /pagamentos/processar/1 (reprocessamento) → 409 SITUACAO_INVALIDA
5. GET /pedidos sem token → 401 NAO_AUTENTICADO

4.3 Multicanalidade com canalPedido

O campo canalPedido (enum CanalAtendimento: APP, TOTEM, BALCAO, PICKUP, WEB) é obrigatório na criação do pedido e persistido no banco. A listagem suporta filtro via query param (?canalPedido=TOTEM), garantindo rastreabilidade por canal sem joins adicionais.

4.4 Validações e Padrão de Erro

Código HTTP	Erro	Quando ocorre
400	CAMPOS_OBRIGATORIOS	Campo obrigatório ausente no body
401	NAO_AUTENTICADO	Token JWT ausente ou inválido
401	CREDENCIAIS_INVALIDAS	Email ou senha incorretos
404	CLIENTE_NAO_ENCONTRADO	clienteId não existe
404	PRODUTO_NAO_ENCONTRADO	produtoId não existe
404	PEDIDO_NAO_ENCONTRADO	id de pedido não existe
409	ESTOQUE_INSUFICIENTE	Quantidade solicitada maior que saldo
409	SITUACAO_INVALIDA	Pagamento de pedido já processado

4.5 Persistência e Migrations

O schema é versionado via Flyway (V1_criar_tabelas.sql), garantindo estrutura reproduzível em qualquer ambiente. No modo H2, as tabelas usam BIGINT GENERATED BY DEFAULT AS IDENTITY (padrão SQL:2003). No modo PostgreSQL, usa BIGSERIAL. O mapeamento ORM reflete fielmente o DER com @ManyToOne, @JoinColumn e @OneToMany.

5. SEGURANÇA, LGPD, LOGS E AUDITORIA

5.1 Autenticação e Autorização

A autenticação é implementada via JWT (stateless). O FiltroJwt (OncePerRequestFilter) intercepta cada requisição, valida o token e popula o SecurityContext. Requisições sem token são rejeitadas com 401 (NAO_AUTENTICADO) via AuthenticationEntryPoint customizado.

O token JWT contém o e-mail (subject) e o perfil (claim "perfil"), com validade de 24h. Limitação: nenhum endpoint aplica @PreAuthorize — qualquer usuário autenticado acessa qualquer rota protegida. Listado como melhoria prioritária.

5.2 LGPD (mínimo)

- Finalidade: dados do cliente têm finalidade exclusiva de identificação para o pedido.
- Minimização: não são coletados dados sensíveis ou desnecessários.
- Consentimento: campos aceiteLgpd (booleano) e dataAceiteLgpd (timestamp) registram o consentimento.

5.3 Armazenamento Seguro

Senhas armazenadas com hash BCrypt via PasswordEncoder do Spring Security. O campo senha nunca é incluído nas respostas da API. A seed de desenvolvimento aplica a senha já hasheada via BCrypt no script SQL de migração.

5.4 Logs e Auditoria

Não implementado nesta versão. O projeto utiliza logs técnicos padrão do Spring Boot em console. Proposta futura: tabela de log_auditoria registrando ação, entidade afetada, usuário e timestamp para eventos críticos.

6. PLANO DE TESTES (API)

6.1 Evidência e Reprodutibilidade

A validação foi realizada com a API em execução local e confirmada em evidência real. A coleção Postman está em /postman/Raizes_Pedro_4710499.postman_collection.json. A requisição T01 executa script que armazena o token na variável {{token}}.

6.2 Cobertura de Cenários (13 cenários)

ID	Cenário	Endpoint	Esperado	Evidência
T01	Login válido	POST /autenticacao/login	200 + token JWT	Auth/T01
T02	Registro de usuário	POST /autenticacao/registro	201 + mensagem	Auth/T02
T03	Criar pedido válido	POST /pedidos	201 + pedidoId	Pedidos/T03
T04	Listar pedidos	GET /pedidos	200 + lista	Pedidos/T04
T05	Buscar pedido por ID	GET /pedidos/{id}	200 + detalhes	Pedidos/T05
T06	Pagamento aprovado	POST /pagamentos/processar/{id}	200 + APROVADO	Pagamento/T06
T07	Acesso sem token	GET /pedidos	401 NAO_AUTENTICADO	Auth/T07
T08	Login senha errada	POST /autenticacao/login	401 CREDENCIAIS_INVALIDAS	Auth/T08
T09	Produto inexistente	POST /pedidos	404 PRODUTO_NAO_ENCONTRADO	Pedidos/T09
T10	Reprocessamento pag.	POST /pagamentos/processar/{id}	409 SITUACAO_INVALIDA	Pagamento/T10
T11	Campo obrig. ausente	POST /pedidos	400 CAMPOS_OBRIGATORIOS	Pedidos/T11
T12	Estoque insuficiente	POST /pedidos	409 ESTOQUE_INSUFICIENTE	Pedidos/T12
T13	Pagamento recusado	POST /pagamentos/processar/{id}	200 + RECUSADO; CANCELADO	Pagamento/T13

6.3 Evidências de Execução (Confirmadas em Ambiente Real)

Os cenários abaixo foram executados com a API rodando localmente (H2 + Java 21 + Maven 3.9.6) e produziram os resultados esperados:

T01 – Login válido (200)

```
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTUzNCJ9...", "perfil": "ADMIN" }
```

T03 – Criar pedido (201)

```
{ "pedidoId":1,"situacao":"AGUARDANDO_PAGAMENTO","canalPedido":"APP","valorTotal":37.80 }
```

T06 – Pagamento aprovado (200)

```
{ "situacaoPagamento":"APROVADO","mensagem":"Pagamento aprovado com sucesso.,"idTransacao":"6dd6f0c0-..." }
```

T07 – Sem token (401)

```
{ "erro":"NAO_AUTENTICADO","mensagem":"Token ausente ou invalido." }
```

T09 – Produto inexistente (404)

```
{ "erro":"PRODUTO_NAO_ENCONTRADO","mensagem":"Produto 9999 nao encontrado." }
```

T10 – Reprocessamento (409)

```
{ "erro":"SITUACAO_INVALIDA","mensagem":"Este pedido nao esta aguardando pagamento." }
```

T11 – Campo ausente (400)

```
{ "erro":"CAMPOS_OBRIGATORIOS","mensagem":"Campos obrigatorios ausentes: [unidadeId, canalPedido, formaPagamento, itens]" }
```

T12 – Estoque insuficiente (409)

```
{ "erro":"ESTOQUE_INSUFICIENTE","mensagem":"Estoque insuficiente para o produto 1" }
```

T13 – Pagamento recusado (200)

```
{ "situacaoPagamento":"RECUSADO","novaSituacaoPedido":"CANCELADO","mensagem":"...valor acima do limite..." }
```

6.4 Cenário Não Coberto

O cenário de acesso com perfil sem permissão (403) não foi implementado, pois não há restrição por perfil via `@PreAuthorize`. Declarado como limitação e melhoria prioritária.

7. ENTREGA TÉCNICA E DOCUMENTAÇÃO

7.1 Organização do Repositório

```
4710499_raizes/ (raiz do repositório GitHub)
src/main/java/com/nordeste/raizes/
  api/controller/          Controllers REST + ConfiguracaoSwagger
  dominio/modelo/          Entidades JPA
```

dominio/enumeracao/	Enums de dominio
infraestrutura/	Seguranca, repositórios, pagamento mock
src/main/resources/	
application.properties	Configuracao H2 (demo) ou PostgreSQL (producao)
db/migration/V1__criar_tabelas.sql	
docs/	Documentacao PDF
postman/	Colecao 13 cenários de teste
pom.xml	Maven (Java 21, Spring Boot 3.2.5)
.gitignore	
README.md	

7.2 Swagger/OpenAPI

A documentação interativa da API está disponível em <http://localhost:8080/swagger-ui.html> após inicialização local. O console H2 (banco de dados em tempo real) está em <http://localhost:8080/h2-console> (URL: jdbc:h2:mem:raizes_db, user: sa, senha: vazia).

7.3 README

O README.md contém: tecnologias utilizadas, pré-requisitos, dois modos de execução (H2 demo e PostgreSQL), comando de execução, URL do Swagger, tabela de endpoints, fluxo principal de teste e tabela dos 13 cenários.

7.4 Adaptações para Execução Local (Modo Demo H2)

Para permitir a execução do projeto sem instalação do PostgreSQL, foram realizadas as seguintes adaptações no código, mantendo a lógica de negócio, as regras e a arquitetura intactas:

- pom.xml: substituída a dependência org.postgresql:postgresql pela com.h2database:h2 (scope runtime); removida flyway-database-postgresql (desnecessária para H2); Java 22 → 21.
- application.properties: datasource redirecionada para jdbc:h2:mem:raizes_db;MODE=PostgreSQL; H2 Console habilitado em /h2-console; dialect H2Dialect configurado.
- V1__criar_tabelas.sql: BIGSERIAL (PostgreSQL) substituído por BIGINT GENERATED BY DEFAULT AS IDENTITY (padrão SQL:2003, suportado pelo H2); demais tipos e constraints mantidos.

- ConfiguracaoSeguranca.java: adicionada liberação de /h2-console/** no filtro de segurança; desativado X-Frame-Options para permitir a interface iframe do H2 Console.

Confirmação em execução real: a API subiu em 3,2 segundos com Java 21.0.7 (PyCharm JBR) + Maven 3.9.6, e todos os cenários de teste foram validados conforme evidenciado na Seção 6.3.

Para voltar ao modo PostgreSQL: restaurar as dependências originais no pom.xml e ajustar o application.properties com as credenciais do banco local.

7.5 Repositório GitHub

O código-fonte completo do projeto, incluindo a documentação PDF e a coleção Postman, está disponível no seguinte repositório:

Repositorio GitHub: <https://github.com/PedroMesquita7/raizes-nordeste-backend.git>

Para clonar e executar: git clone <URL> → cd 4710499_raizes → (configurar Java e Maven conforme README.md) → mvn spring-boot:run

8. CONCLUSÃO

Este projeto implementou uma API REST funcional para o sistema de pedidos da Rede Raízes do Nordeste, cobrindo o fluxo crítico completo de criação de pedido, validação de estoque, processamento de pagamento mock e atualização de situação, com autenticação JWT e documentação via Swagger. O fluxo foi validado — em execução real na máquina — com 13 cenários reproduzíveis via coleção Postman, superando o mínimo exigido pelo roteiro (10 cenários, 6 positivos e 4 negativos).

A modelagem produzida — DER, Diagrama de Classes e Diagrama de Casos de Uso — reflete fielmente as sete entidades JPA implementadas e os endpoints reais do sistema. A arquitetura adota separação em camadas (api, dominio, infraestrutura), com execução validada em dois modos de banco de dados: H2 em memória (demo, sem instalação) e PostgreSQL (produção).

O gateway de pagamento mock (GatewayPagamentoMock) simula aprovação para valores até R\$ 500,00 e recusa para valores superiores. As validações de negócio seguem padrão de erro consistente, cobrindo campos ausentes (400), autenticação (401), entidades inexistentes (404) e conflitos de regra de negócio (409).

As lacunas assumidas explicitamente: (i) ausência de @PreAuthorize por perfil; (ii) sem camada de logs/auditoria; (iii) programa de fidelização não implementado. Melhorias prioritárias: (1) @PreAuthorize nos endpoints sensíveis; (2) extrair camada de serviços; (3) tabela de auditoria.

REFERÊNCIAS

- ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação – trabalhos acadêmicos – apresentação. Rio de Janeiro: ABNT, 2011.
- BRASIL. Lei n.º 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.
- SPRING. Spring Boot Reference Documentation. Disponível em: <https://docs.spring.io/spring-boot/>. Acesso em: 25 jun. 2026.
- SPRING. Spring Security Reference Documentation. Disponível em: <https://docs.spring.io/spring-security/reference/>. Acesso em: 25 jun. 2026.
- POSTMAN. Postman Learning Center. Disponível em: <https://learning.postman.com/>. Acesso em: 25 jun. 2026.
- JWT.IO. Introduction to JSON Web Tokens. Disponível em: <https://jwt.io/introduction>. Acesso em: 25 jun. 2026.
- H2 DATABASE ENGINE. H2 Console and Compatibility Modes. Disponível em: <https://h2database.com/html/main.html>. Acesso em: 25 jun. 2026.
- UNINTER. **Roteiro de Atividade Prática de Projeto Multidisciplinar – Trilha Back-End.** Curitiba: Centro Universitário Internacional – Uninter, 2026.

DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

Em conformidade com as orientações da Uninter para trabalhos acadêmicos, declaro que durante a elaboração deste projeto utilizei o assistente de Inteligência Artificial Claude (Anthropic) como ferramenta de apoio nas seguintes etapas:

A.1 Etapas em que a IA foi utilizada

- Estruturação e escrita do documento: organização das seções, redação dos textos de análise de requisitos, modelagem, implementação, segurança/LGPD e conclusão.
- Geração do código-fonte: arquivos Java (controllers, modelos, repositórios, segurança, migrations Flyway) gerados com base nos requisitos do roteiro.
- Organização da coleção Postman: geração do JSON da coleção com os 13 cenários de teste.
- Execução e depuração local: identificação do Java 21 (PyCharm JBR), download do Maven 3.9.6, adaptação do projeto para banco H2 em memória e validação de todos os endpoints em execução real.

A.2 Ferramenta Utilizada

Ferramenta: Claude (Anthropic) — claude.ai | Versão: Claude Sonnet 4.6 | **Período de uso: junho de 2026**

Declaro que todos os textos e código gerados com auxílio de IA foram revisados pelo autor, e que a IA atuou como assistente de desenvolvimento, conforme declaração exigida pelo roteiro.